

MLOps Assignment 3

End-to-End Hugging Face Classification & Docker Deployment

Github link:

<https://github.com/Layaa-V/MLOps-LayaaVishwakarma-M25CSA017/tree/Assignment-3>

Hugging face model link:

<https://huggingface.co/Layaa-V/FineTuned-Bert-Classifier>

In this Assignment we take an experimental Jupyter, convert it into professional, production-ready code, train it, and deploy it using Docker and cloud platforms like GitHub and Hugging Face.

Task 1: Acquiring the Baseline Code

- We downloaded the provided Google Colab notebook (ML_DL_Ops_Ass_3_Fine_Tuning_Classification.ipynb).
- A dedicated local directory was created to serve as the main project hub, keeping all datasets, scripts, and Dockerfiles cleanly organized in one place.

Task 2: Creating the Development Environment (Docker)

- The Purpose of Docker was, instead of installing machine learning libraries directly onto a personal computer (which causes version conflicts), we used Docker. Docker creates an isolated "virtual computer" (container) where the code runs exactly the same way on any machine.
 - We wrote a Dockerfile which ensured the container could automatically talk to the computer's GPU to speed up training.
 - A requirements.txt file was created (listing tools like transformers, torch, and scikit-learn) and copied into the container.
 - The command used for writing docker file: "nano dockerfile".
 - For viewing the above created dockerfile, we can use the command: "cat dockerfile".
 - For building the image from this dockerfile the command used is: "docker build -t ass3_image ." (Where ass3_image is the name of image we are building).
-

Task 3: Converting the Notebook to Python Scripts

The notebook code was cleaned and separated into distinct, purpose-built files:

- `train.py`: Dedicated strictly to downloading the data, preparing it, and teaching the model.
- `eval.py`: Dedicated strictly to testing the model's accuracy on unseen data.

Task 4: Load Models from Hugging Face

- We selected `distilbert-base-cased` from the Hugging Face library. It is a "distilled" (compressed) version of the massive BERT language model.
- The text reviews were passed through a DistilBERT Tokenizer, which converted the English words into numbers (tensors) that the neural network could understand.

Task 5: Train Model Using Trainer API

We used the Hugging Face Trainer class to handle the complex training loop. The model was trained over 3 epochs (full passes of the data) using a batch size of 10 and a learning rate of $5e-5$.

Task 6: Model Evaluation

Immediately after training, the model was tested against new, unseen set of data.

Task 7: Pushing Model to Hugging Face Hub

The `trainer.push_to_hub()` command uploaded the heavy neural network weights, the tokenizer dictionary, and the training configuration directly to a public Hugging Face profile.

Task 8: Re-Evaluating Model from the Hugging Face repo.

- `eval.py` was executed to ensure the uploaded model wasn't corrupted.
 - Instead of reading the model from the local hard drive, the script downloaded the trained model directly from the Hugging Face Hub over the internet, loaded the saved test dataset, and generated a final classification report.
-

Task 9: The Final Production Docker Image (Evaluation Only)

Dockerfile.eval: We created a second lightweight Dockerfile. It was configured to automatically run eval.py the moment the container booted up. This represents a highly efficient, deployable production asset.

Task 10: Pushing Everything to GitHub

- **The .gitignore File:** .gitignore file handles the massive PyTorch model weights from Git. This prevented the terminal from freezing and ensured we didn't violate GitHub's strict file-size limits.
 - The final configured files were pushed to a github repository, the link of which is provided at the start of the report
-